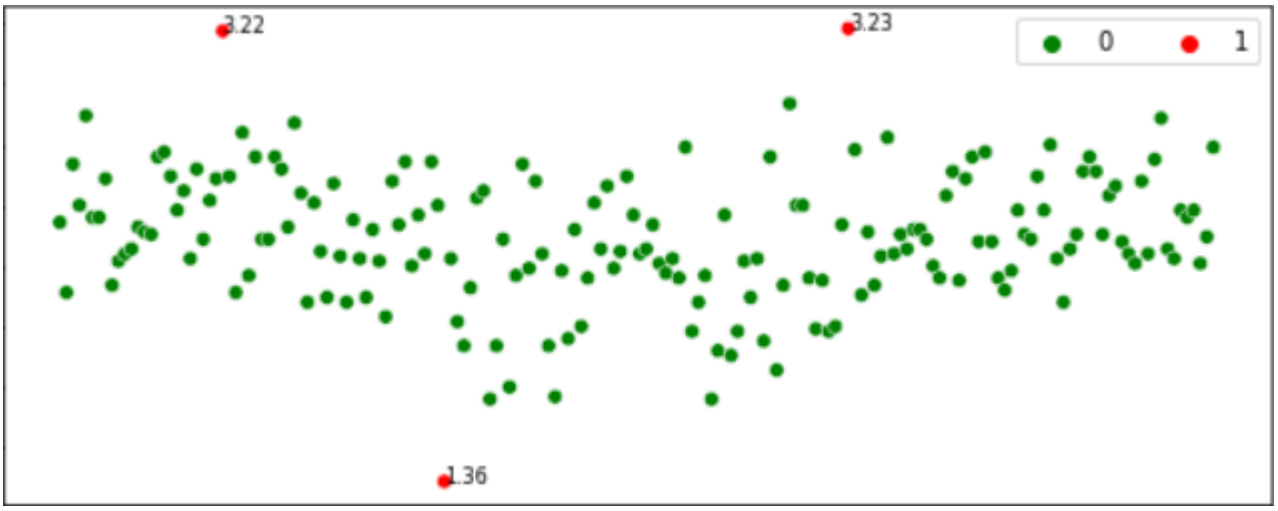
Outliers

Outliers are the values that look different from the other values in the data. Below is a plot highlighting the outliers in ‘red’ and outliers can be seen in both the extremes of data.

This outlier could be the result of many different issues:

- Human error
- Instrument error
- Experimental error
- Intentional creation
- Data processing error
- Sampling error
- Natural outlier



Detecting outliers

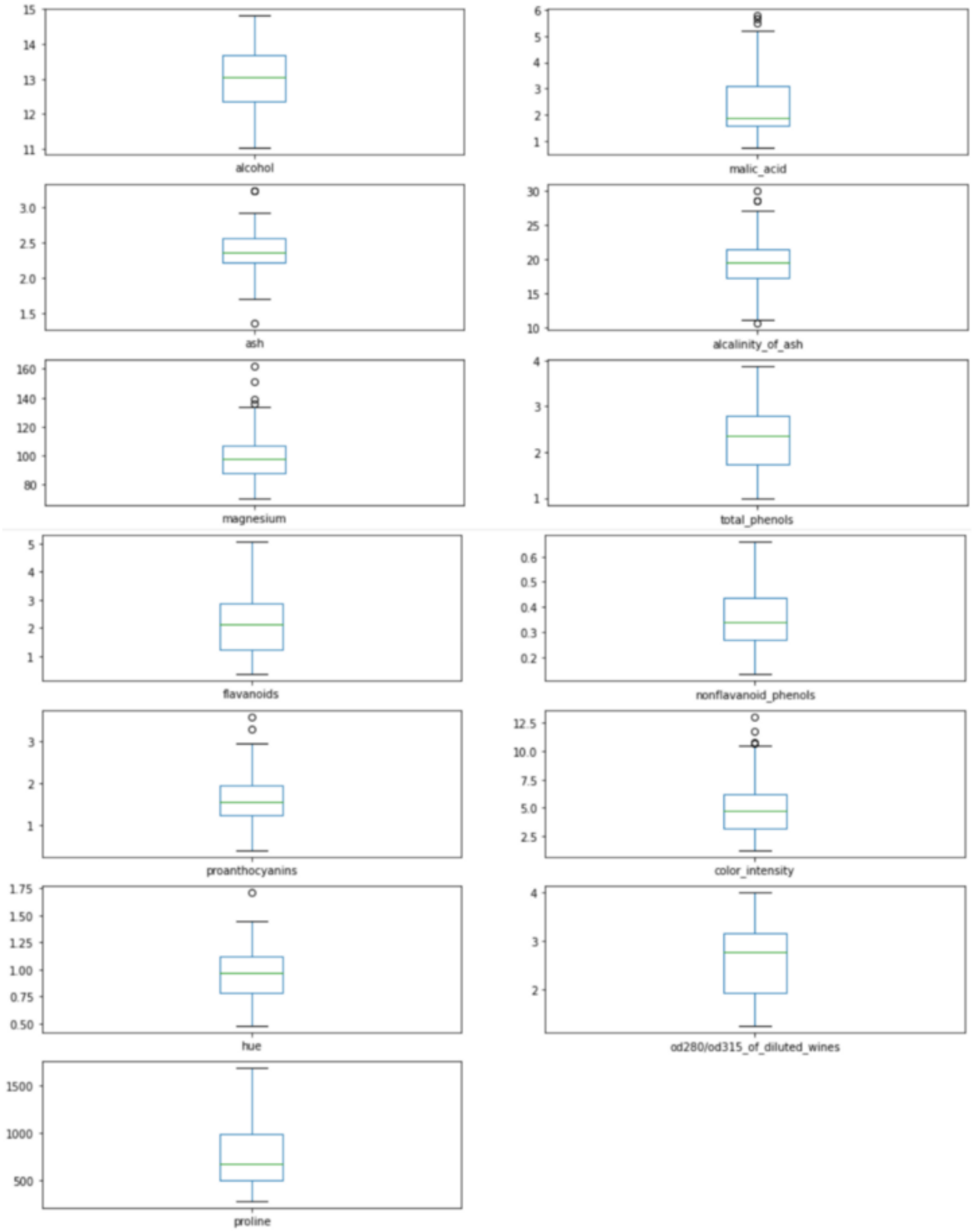
I. Univariate outlier detection

1. Box plots

Box plots are a visual method to identify outliers. Box plots is one of the many ways to visualize data distribution. Box plot plots the q1 (25th percentile), q2 (50th percentile or

median) and q3 (75th percentile) of the data along with *(q1–1.5*(q3-1))* and *(q3+1.5*(q3-q1))*. Outliers, if any, are plotted as points above and below the plot.

```
data.plot(kind="box",subplots=True,layout=(7,2),figsize=(15,20));
```



In the above plot, outliers are shown as points below and above the box plot. ‘alcohol’, ‘total_phenols’, ‘od280/od315_of_diluted_wines’, and ‘proline’ have no outliers.

2. IQR method

IQR method is used by box plot to highlight outliers. IQR stands for interquartile range, which is the difference between q3 (75th percentile) and q1 (25th percentile). The IQR method computes lower bound and upper bound to identify outliers.

*Lower Bound = q1–1.5*IQR*

*Upper Bound = q3+1.5*IQR*

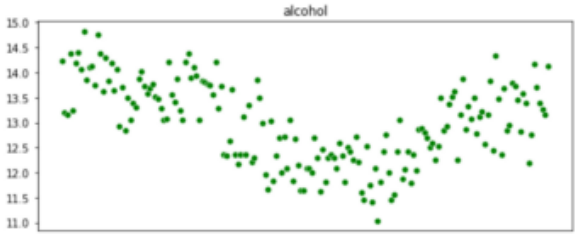
Any value below the lower bound and above the upper bound are considered to be outliers. Below is the implementation of IQR method in Python.

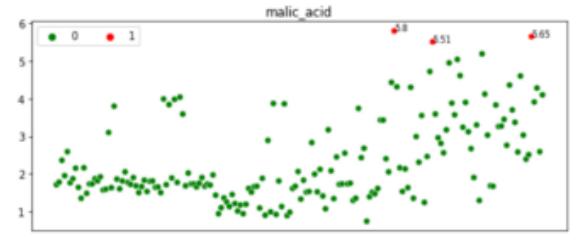
```
#FUNCTION TO IDENTIFY OUTLIERS USING IQR METHOD
def iqr_outlier(x,factor):
    q1 = x.quantile(0.25)
    q3 = x.quantile(0.75)
    iqr = q3 - q1
    min_ = q1 - factor * iqr
    max_ = q3 + factor * iqr
    result_ = pd.Series([0] * len(x))
    result_[((x < min_) | (x > max_))] = 1
    return result_
#SCATTER PLOTS HIGHLIGHTING OUTLIERS CALCULATED USING IQR METHOD
fig, ax = plt.subplots(7, 2, figsize=(20, 30))
row = col = 0
for n,i in enumerate(data.columns):
    if (n % 2 == 0) & (n > 0):
        row += 1
        col = 0
    outliers = iqr_outlier(data[i], 1.5)

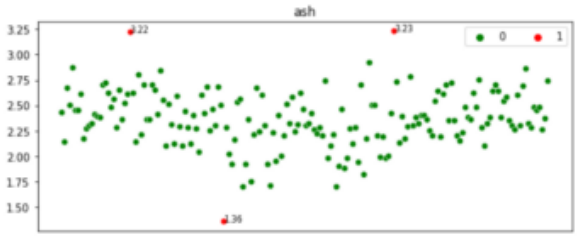
    if sum(outliers) == 0:
        sns.scatterplot(x = np.arange(len(data[i])), y = data[i], ax = ax[row,
col], legend=False, color = 'green')
    else:
        sns.scatterplot(x = np.arange(len(data[i])), y = data[i], ax = ax[row,
col], hue = outliers, palette = ['green','red'])
    for x,y in zip(np.arange(len(data[i]))[outliers == 1], data[i][outliers ==
1]):
        ax[row,col].text(x = x, y = y, s = y, fontsize = 8)
    ax[row,col].set_ylabel("")
```

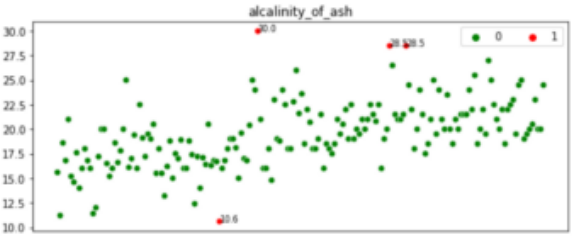
```
    ax[row,col].set_title(i)
    ax[row,col].xaxis.set_visible(False)
    if sum(outliers) > 0:
        ax[row,col].legend(ncol=2)
    col += 1
ax[row,col].axis('off')
plt.show()
```

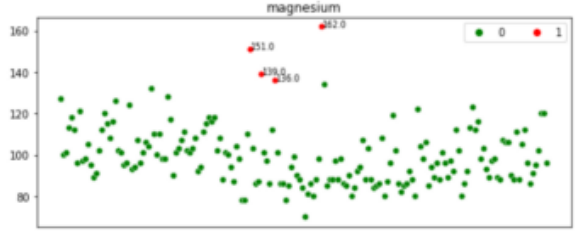
In the below plot, as seen in the box plot, ‘alcohol’, ‘total_phenols’, ‘od280/od315_of_diluted_wines’ and ‘proline’ have no outliers.

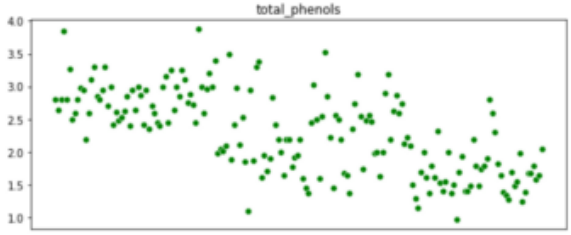

alcohol

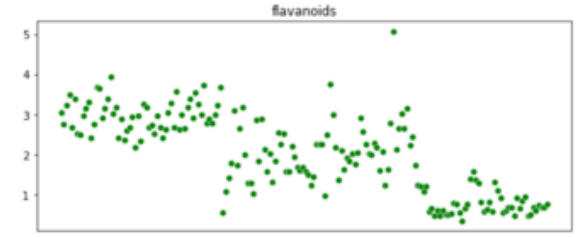

malic_acid
0
1

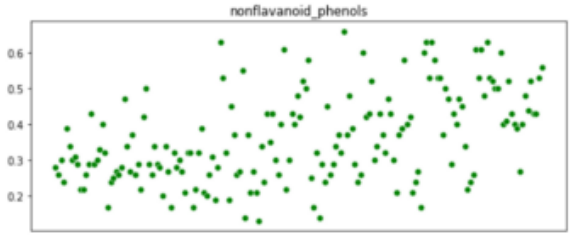

ash
0
1

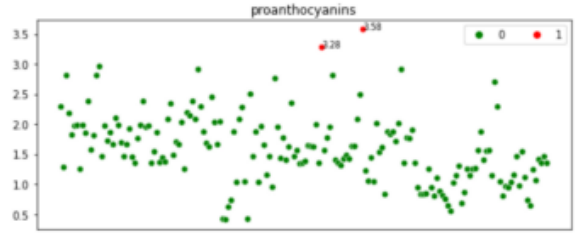

alcalinity_of_ash
0
1

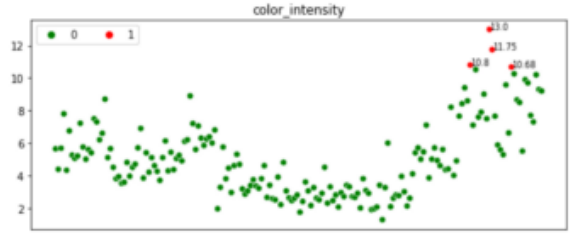

magnesium
0
1

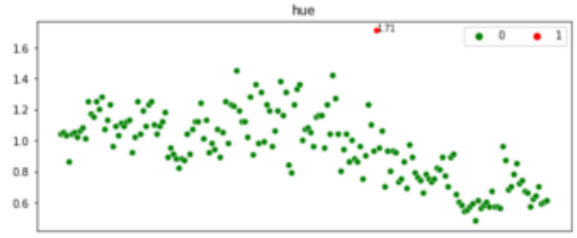

total_phenols

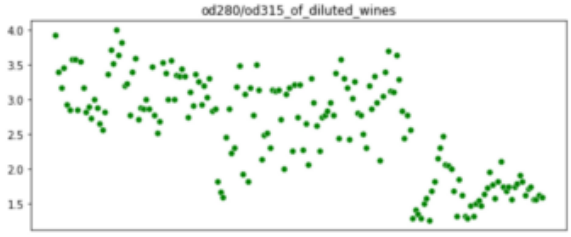

flavanoids

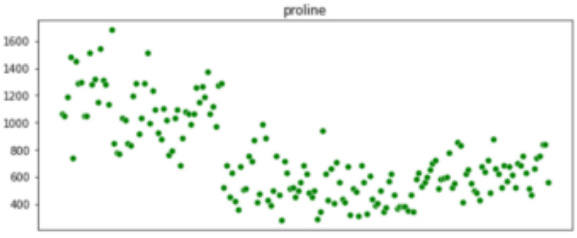

nonflavanoid_phenols


proanthocyanins
0
1


color_intensity
0
1


hue
0
1


od280/od315_of_diluted_wines


proline

3. Z-score method

Z-score method is another method for detecting outliers. This method is generally used when a variable' distribution looks close to Gaussian. Z-score is the number of standard deviations a value of a variable is away from the variable' mean.

Z-Score = (X - mean) / Standard deviation

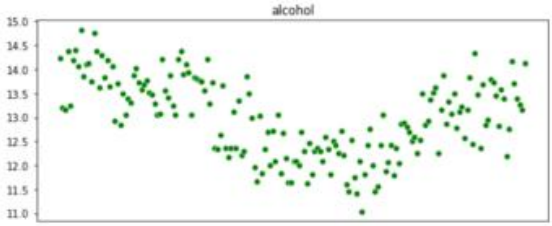
when the values of a variable are converted to Z-scores, then the distribution of the variable is called standard normal distribution with mean=0 and standard deviation=1. The Z-score method requires a cut-off specified by the user, to identify outliers. The widely used lower end cut-off is -3 and the upper end cut-off is +3. The reason behind using these cut-offs is, 99.7% of the values lie between -3 and +3 in a standard normal distribution. Lets look at the implementation of Z-Score method in Python.

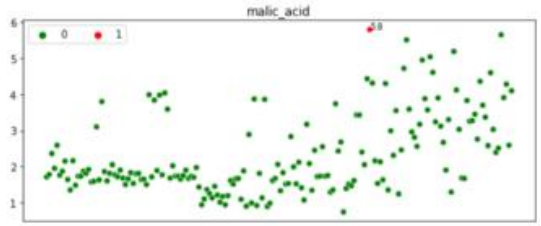
```
#FUNCTION TO DETECT OUTLIERS USING
Z-SCORE METHOD
def zscore_outlier(x,lb,ub):
    zscore = ((x - x.mean()) /
x.std()).copy()
    result_ = pd.Series([0] *
len(x))
    result_[((zscore < lb) | (zscore
> ub))] = 1
    return result_
#PLOTTING A SCATTER PLOT AND
HIGHLIGHTING THE OUTLIERS DETECTED
BY Z-SCORE METHOD
fig, ax = plt.subplots(7, 2,
figsize=(20, 30))
row = col = 0
for n,i in enumerate(data.columns):
    if (n % 2 == 0) & (n > 0):
        row += 1
        col = 0
    outliers =
zscore_outlier(data[i], -3, 3)

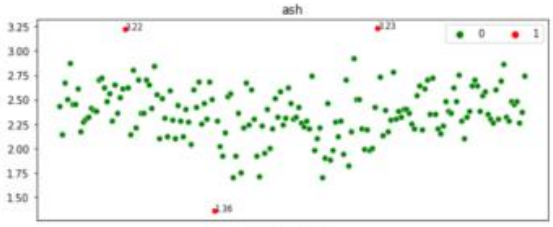
    if sum(outliers) == 0:
        sns.scatterplot(x =
np.arange(len(data[i])), y =
```

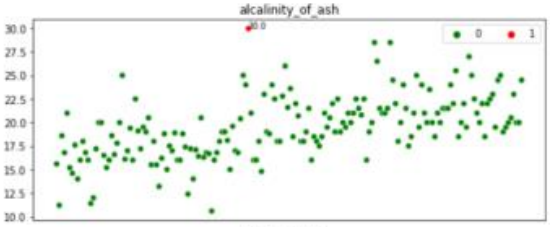
```
data[i], ax = ax[row, col],
legend=False, color = 'green')
    else:
        sns.scatterplot(x =
np.arange(len(data[i])), y =
data[i], ax = ax[row, col], hue =
outliers, palette = ['green','red'])
    for x,y in
zip(np.arange(len(data[i]))[outliers
== 1], data[i][outliers == 1]):
        ax[row,col].text(x = x, y =
y, s = y, fontsize = 8)
    ax[row,col].set_ylabel("")
    ax[row,col].set_title(i)

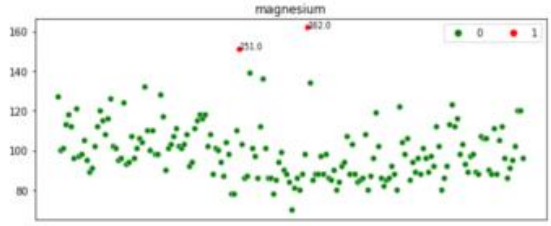
ax[row,col].xaxis.set_visible(False)
    if sum(outliers) > 0:
        ax[row,col].legend(ncol=2)
    col += 1
ax[row,col].axis('off')
plt.show()
```

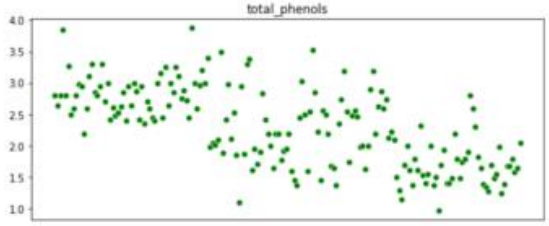

alcohol
15.0
14.5
14.0
13.5
13.0
12.5
12.0
11.5
11.0

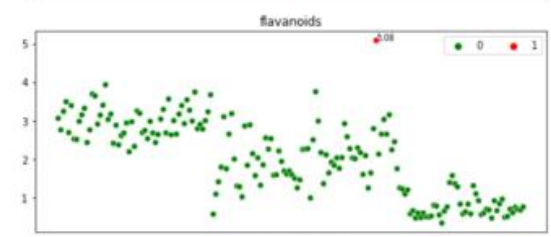

malic_acid
0
1
5.8
6
5
4
3
2
1

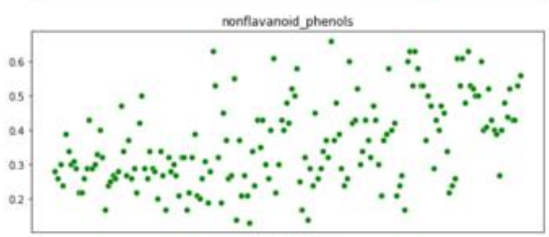

ash
3.22
3.23
1.36
0
1
3.25
3.00
2.75
2.50
2.25
2.00
1.75
1.50

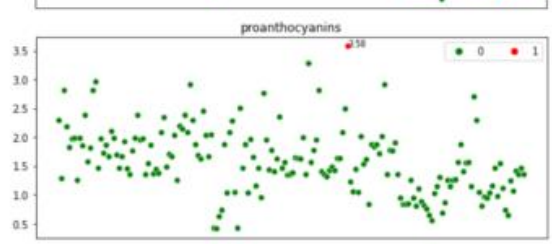

alcalinity_of_ash
30.0
0
1
30.0
27.5
25.0
22.5
20.0
17.5
15.0
12.5
10.0

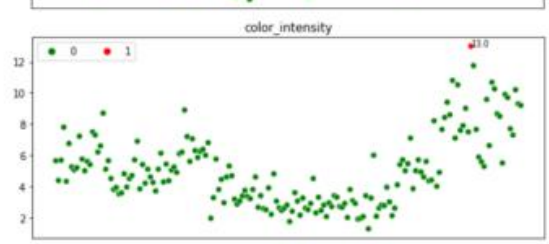

magnesium
162.0
151.0
0
1
160
140
120
100
80

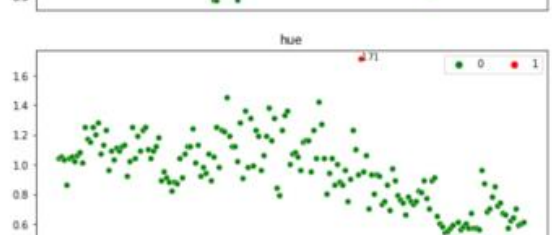

total_phenols
4.0
3.5
3.0
2.5
2.0
1.5
1.0

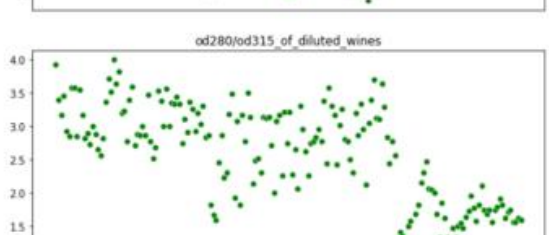

flavanoids
5.08
0
1
5
4
3
2
1

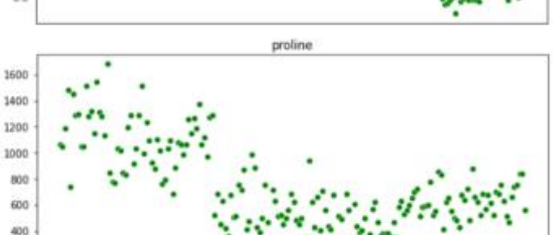

nonflavanoid_phenols
0.6
0.5
0.4
0.3
0.2


proanthocyanins
3.58
0
1
3.5
3.0
2.5
2.0
1.5
1.0
0.5


color_intensity
13.0
0
1
12
10
8
6
4
2


hue
1.71
0
1
1.6
1.4
1.2
1.0
0.8
0.6


od280/od315_of_diluted_wines
4.0
3.5
3.0
2.5
2.0
1.5

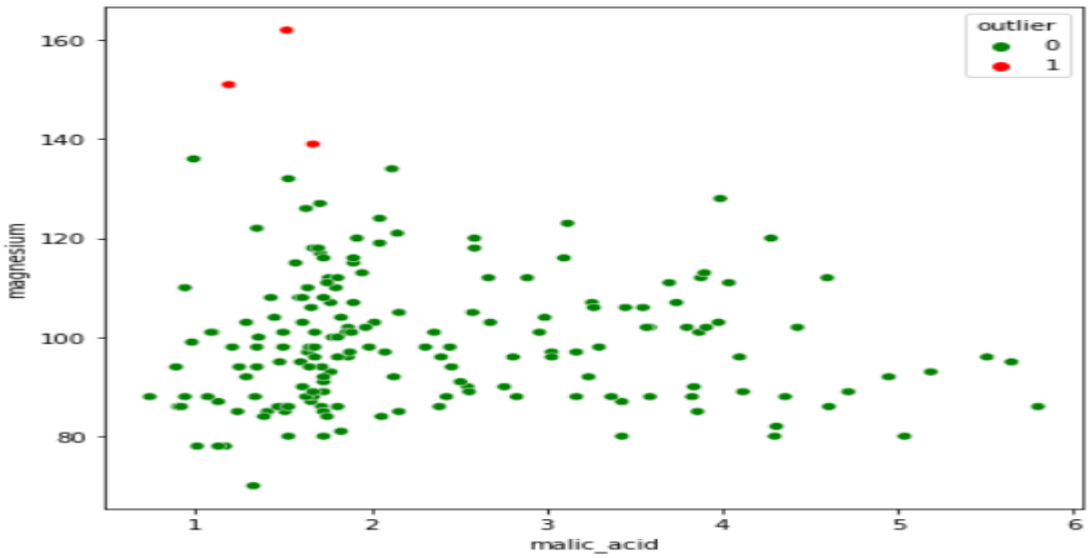

proline
1600
1400
1200
1000
800
600
400

II. Bivariate outlier detection

1. **'Distance from the mean' method** (Multivariate method)

Unlike the previous methods, this method considers multiple variables in a data set to detect outliers. This method calculates the Euclidean distance of the data points from their mean and converts the distances into absolute z-scores. Any z-score greater than the pre-specified cut-off is considered to be an outlier. We'll consider two variables ('malic_acid' and 'magnesium') of the wine dataset for implementing this method in Python using a cut-off of 3.

```
def euclidean_distance_outlier(x,cutoff):
    result_ = pd.Series([0] * len(x))
    data_mean = x.mean() # mean of data
    dist = np.sqrt(np.sum(((x-data_mean) ** 2),axis=1)) #Euclidean distande
    dist_mean = dist.mean() #mean of the distances
    dist_zscore = np.abs((dist - dist_mean) / dist.std())#z-score of the distances
    result_[((dist_zscore > cutoff))] = 1
    return result_
euc_d = data[["malic_acid","magnesium"]].copy()
d['outlier'] = euclidean_distance(d,3)
sns.scatterplot(x="malic_acid",y="magnesium",data=d,hue="outlier",palette=["green","red"])
```

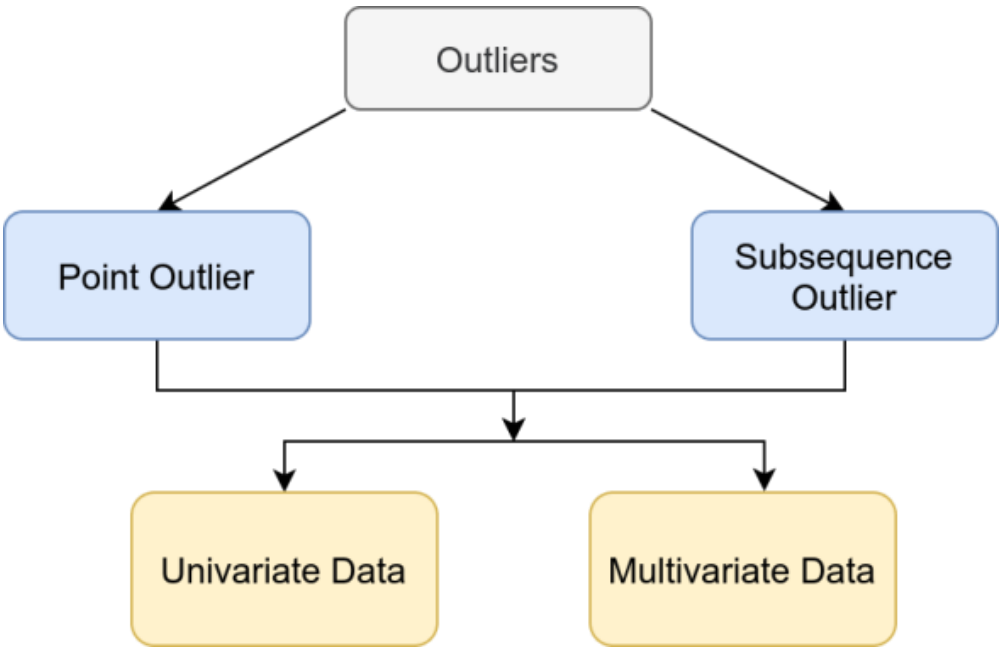


These are few commonly used outlier detection methods in machine learning. Presence of outliers may cause problems during model fitting (esp. linear models) and may also result in inflated error metrics which give higher weights to large errors. Hence, it is necessary to treat outliers before building a machine learning model.

III. Time series outlier detection

In recent years – especially in the area of time series data – many researchers have aimed to detect and analyze unusual, but interesting phenomena. *Fraud detection* is a good example – the main objective is to detect and analyze the outlier itself. These observations are often referred to as anomalies.

The anomaly detection problem for time series is usually formulated as *identifying outlier data points relative to some norm or usual signal*. Take a look at some outlier types:



Point outlier

A point outlier is a datum that behaves unusually in a specific time instance when compared either to the other values in the time series (global outlier), or to its neighboring points (local outlier).

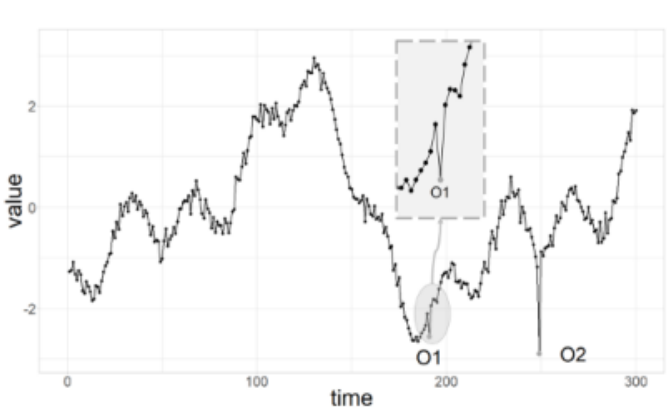
Example: are you aware of the Gamestop frenzy? A slew of young retail investors bought GME stock to get back at big hedge funds, driving the stock price way up. That sudden, short-lived spike that occurred due to an unlikely event is an **additive (point) outlier.** The unexpected growth of a

time-based value in a short period (looks like a sudden spike) comes under additive outliers.

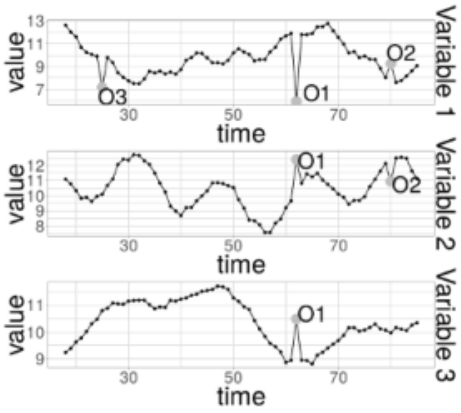


Point outliers can be *univariate* or *multivariate*, depending on whether they affect one or more time-dependent variables, respectively.

Above figure contains two univariate point outliers, O1 and O2, whereas the multivariate time series is composed of three variables in below figure, and has both univariate (O3) and multivariate (O1 and O2) point outliers.



(a) Univariate time series.

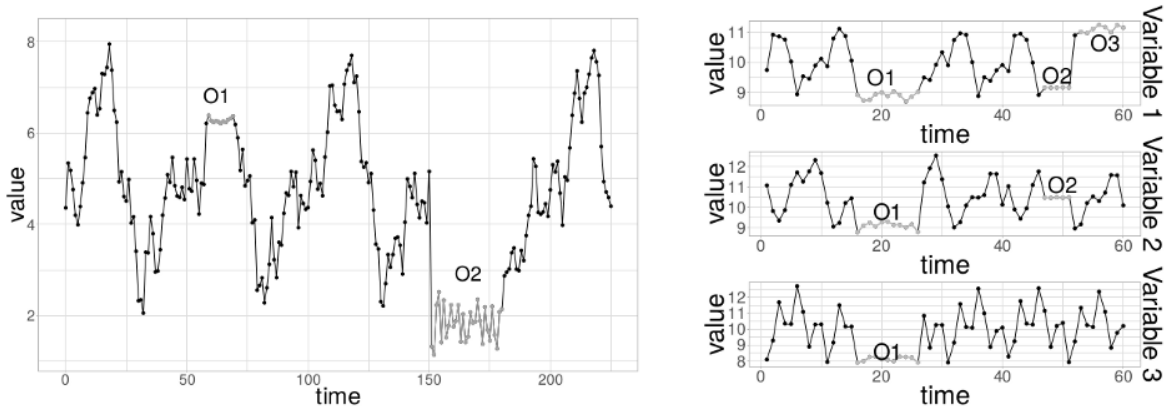


(b) Multivariate time series.

Subsequence outlier

This means consecutive points in time whose joint behavior is unusual, although each observation individually is not necessarily a point outlier. Subsequence outliers can also be global or local, and can affect one (univariate subsequence outlier) or more (multivariate subsequence outlier) time-dependent variables.

Figure below provides an example of univariate (O1 and O2 in Fig. a, and O3 in Fig. b) and multivariate (O1 and O2 in Fig. 2b) subsequence outliers. Note that the latter does not necessarily affect all the variables (e.g., O2 in Fig. b).



(a) Univariate time series. (b) Multivariate time series.

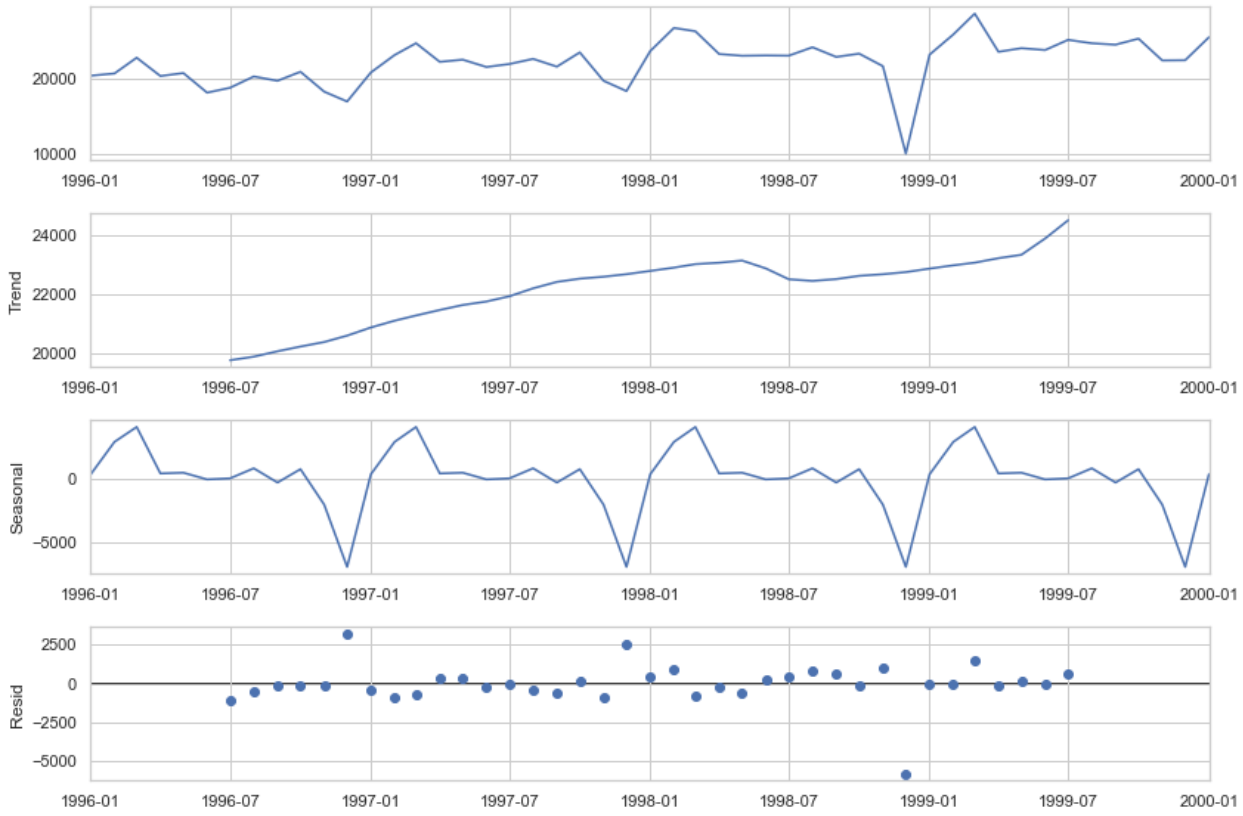
Anomaly detection techniques in time series data

There are few techniques that analysts can employ to identify different anomalies in data. It starts with a basic statistical decomposition and can work up to autoencoders. Let's start with the basic one, and understand how and why it's useful.

STL decomposition

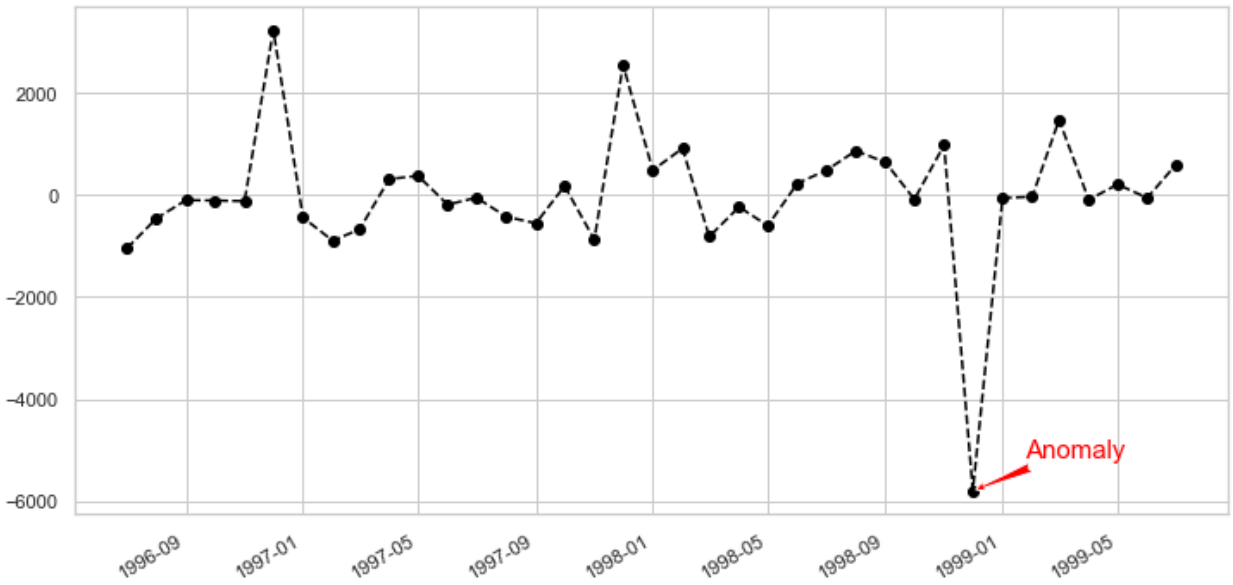
STL stands for seasonal-trend decomposition procedure based on LOESS. This technique gives you the ability to split your time series signal into three parts: **seasonal, trend, and residue**.

It works for seasonal time-series, which is also the most popular type of time series data. To generate an STL-decomposition plot, we just use the ever-amazing *statsmodels* to do the heavy lifting for us.



If we analyze the deviation of **residue** and introduce some threshold for it, we'll get an anomaly detection algorithm. To implement this, we only need the residue data from the decomposition.

```
plt.rc('figure',figsize=(12,6))
plt.rc('font',size=15)
fig, ax = plt.subplots()
x = result.resid.index
y = result.resid.values
ax.plot_date(x, y, color='black',linestyle='--')
ax.annotate('Anomaly', (mdates.date2num(x[35]), y[35]), xytext=(30, 20),
            textcoords='offset points',
color='red',arrowprops=dict(facecolor='red',arrowstyle='fancy'))
fig.autofmt_xdate()
plt.show()
```



Classification and Regression Trees (CART)

We can utilize the power and robustness of Decision Trees to identify outliers/anomalies in time series data.

- First, you can use supervised learning to teach trees to classify anomaly and non-anomaly data points. In order to do that, we'd need to have labeled anomaly data points, which you won't find often outside of toy datasets.
- Unsupervised is what you need! We can use the Isolation Forest algorithm to predict whether a certain point is an outlier or not, without the help of any labeled dataset. Let's see how.

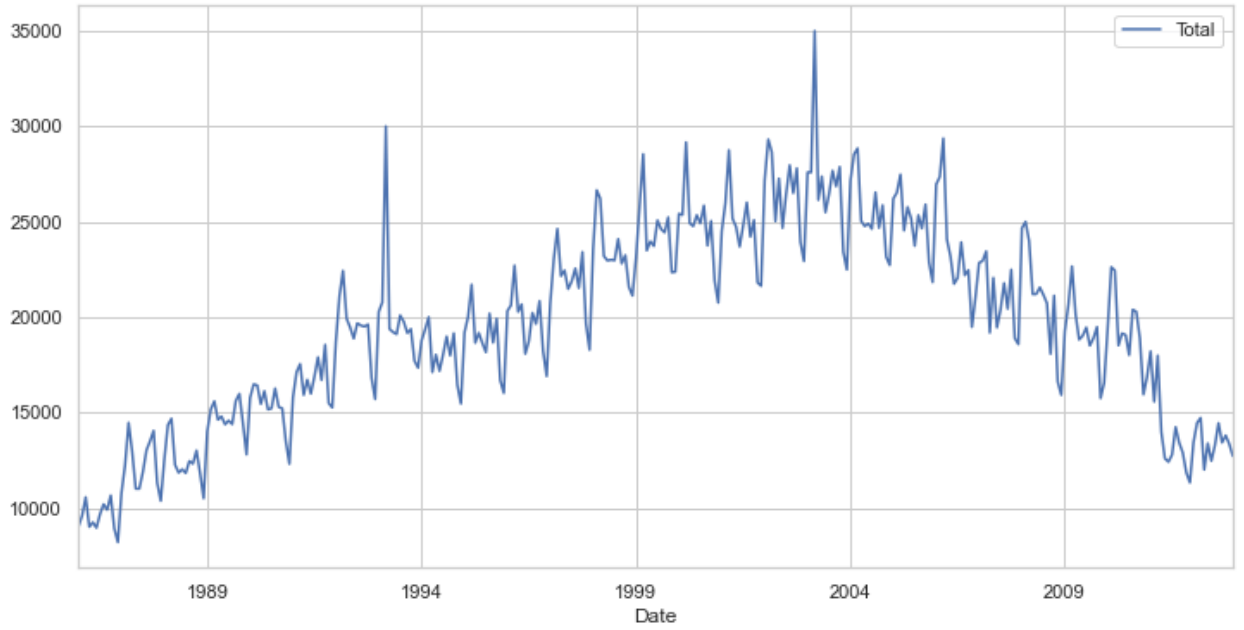
The main idea, which is different from other popular outlier detection methods, is that Isolation Forest explicitly identifies anomalies instead of profiling normal data points. Isolation Forest, like any tree ensemble method, is based on decision trees.

In other words, Isolation Forest detects anomalies purely based on the fact that anomalies are data points that are few and different. The anomalies isolation is implemented without employing any distance or density measure.

- When applying an IsolationForest model, we set contamination = outliers_fraction, that is telling the model what proportion of outliers are present in the data. This is a trial/error metric.
- Fit and predict (data) performs outlier detection on data, and returns 1 for normal, -1 for the anomaly.
- Finally, we visualize anomalies with the Time Series view.

Let's do it step by step. First, visualize the time series data:

```
plt.rc('figure',figsize=(12,6))
plt.rc('font',size=15)
catfish_sales.plot()
```

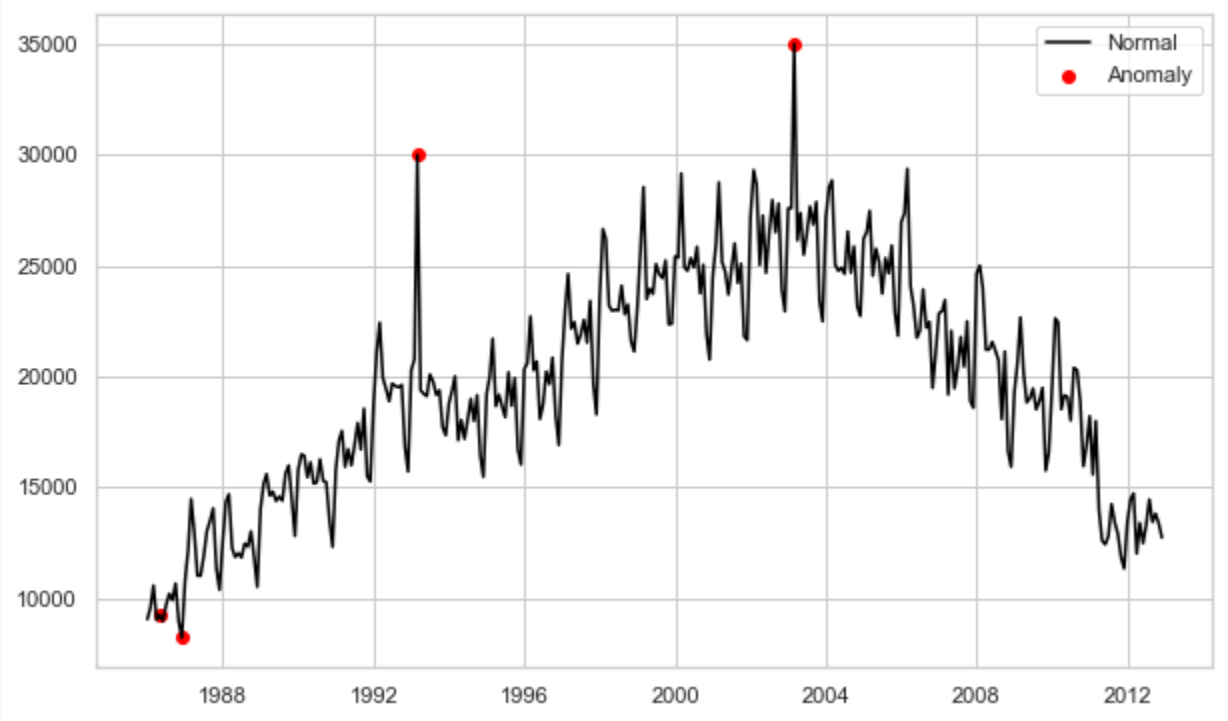


Next, we need to set some parameters like the outlier fraction, and train our IsolationForest model. We can utilize the super useful scikit-learn to implement the Isolation Forest algorithm.

```
outliers_fraction = float(.01)
scaler = StandardScaler()
np_scaled = scaler.fit_transform(catfish_sales.values.reshape(-1, 1))
data = pd.DataFrame(np_scaled)
# train isolation forest
model =  IsolationForest(contamination=outliers_fraction)
model.fit(data)
```

Lastly, we need to visualize how the prediction was.

```
catfish_sales['anomaly'] = model.predict(data)
# visualization
fig, ax = plt.subplots(figsize=(10,6))
a = catfish_sales.loc[catfish_sales['anomaly'] == -1, ['Total']] #anomaly
ax.plot(catfish_sales.index, catfish_sales['Total'], color='black', label =
'Normal')
ax.scatter(a.index,a['Total'], color='red', label = 'Anomaly')
plt.legend()
plt.show();
```



Detection using Forecasting

Anomaly detection using Forecasting is based on an approach that several points from the past generate a forecast of the next point with the addition of some random variable, which is usually white noise.

As you can imagine, forecasted points in the future will generate new points and so on. Its obvious effect on the forecast horizon – the signal gets smoother.

The difficult part of using this method is that you should select the number of differences, number of autoregressions, and forecast error coefficients.

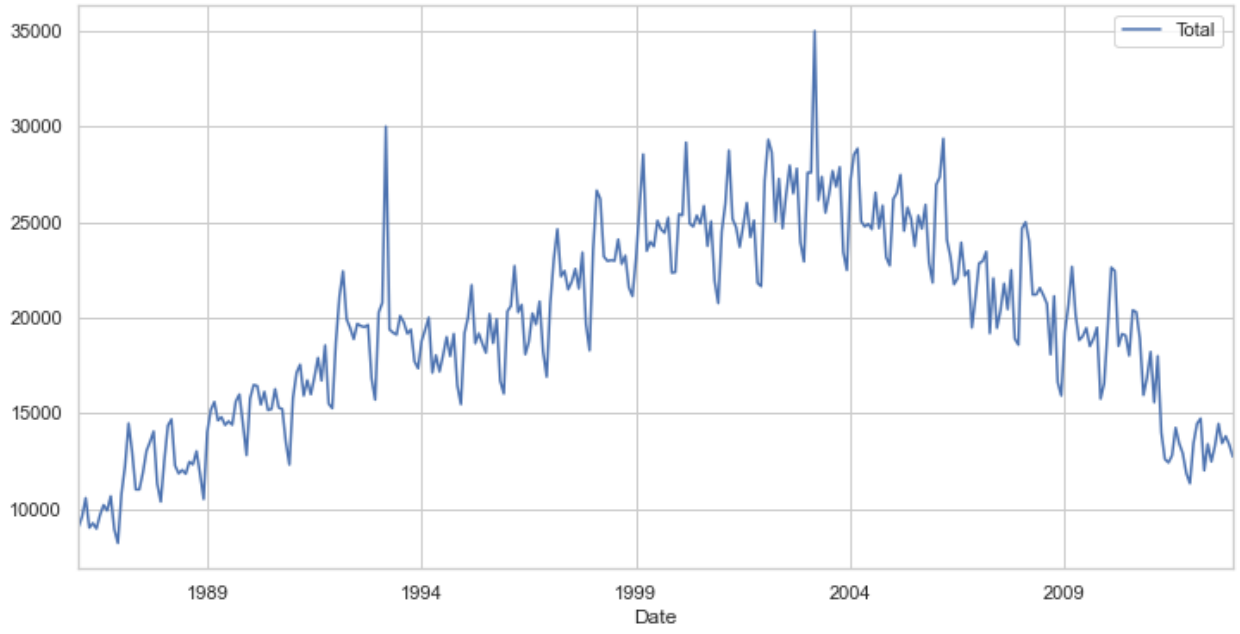
Each time you work with a new signal, you should build a new forecasting model.

Another obstacle is that your signal should be stationary after differencing. In simple words, it means your signal shouldn't be dependent on time, which is a significant constraint.

We can utilize different forecasting methods such as Moving Averages, Autoregressive approach, and ARIMA with its different variants. The procedure for detecting anomalies with ARIMA is:

- Predict the new point from past datums and find the difference in magnitude with those in the training data.
- Choose a threshold and identify anomalies based on that difference threshold. That's it!

To test this technique, we're gonna use a popular module in time series called ***fbprophet***. This module specifically caters to stationarity and seasonality, and can be tuned with some hyper-parameters.




```
from fbprophet import Prophet
```

Now let's define the forecasting function. An important thing to note here is that *fbprophet* will add some additional metrics as features, in order to help identify anomalies better. For example, the predicted time series variable (by the model), the upper and lower limit of the target time series variable, and the trend metric.

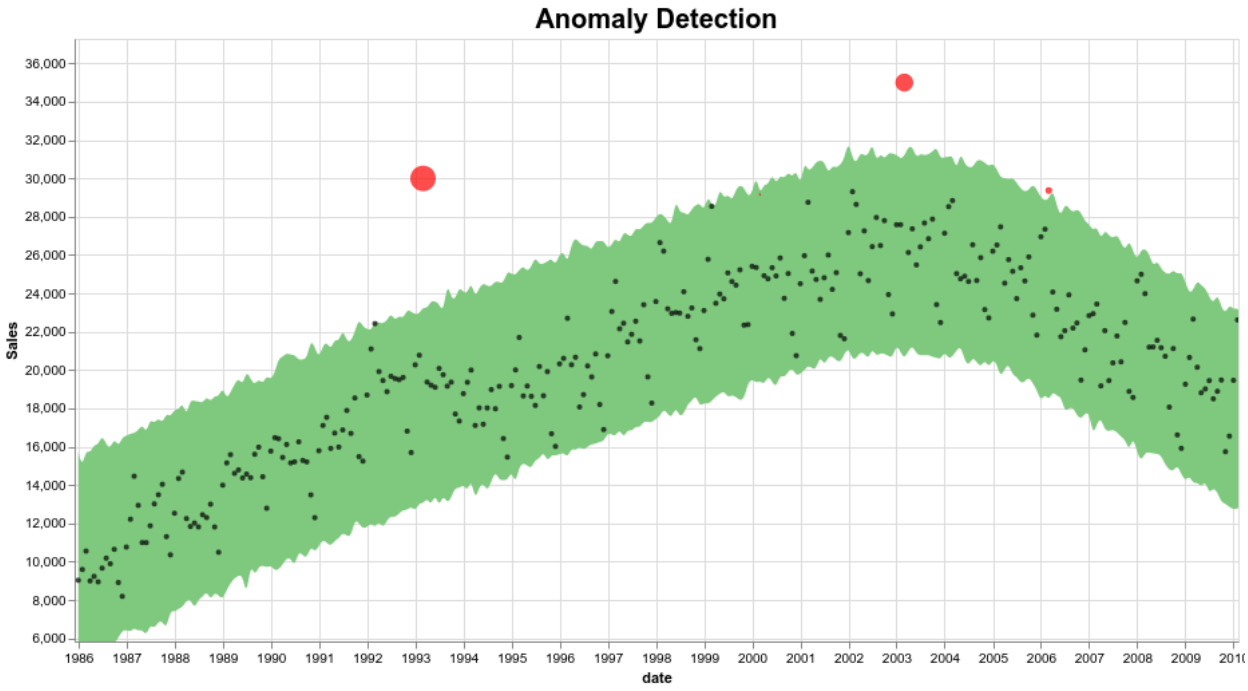
```
def fit_predict_model(dataframe, interval_width = 0.99, changepoint_range =
0.8):
   m = Prophet(daily_seasonality = False, yearly_seasonality = False,
weekly_seasonality = False,
               seasonality_mode = 'additive',
               interval_width = interval_width,
               changepoint_range = changepoint_range)
   m = m.fit(dataframe)
   forecast = m.predict(dataframe)
   forecast['fact'] = dataframe['y'].reset_index(drop = True)
   return forecast

pred = fit_predict_model(t)
```

We now have to push the *pred* variable to another function, which will detect anomalies based on a threshold of lower and upper limit in the time series variable.

```
def detect_anomalies(forecast):
   forecasted = forecast[['ds','trend', 'yhat', 'yhat_lower', 'yhat_upper',
'fact']].copy()
forecasted['anomaly'] = 0
   forecasted.loc[forecasted['fact'] > forecasted['yhat_upper'], 'anomaly'] =
1
   forecasted.loc[forecasted['fact'] < forecasted['yhat_lower'], 'anomaly'] =
-1
#anomaly importances
   forecasted['importance'] = 0
   forecasted.loc[forecasted['anomaly'] ==1, 'importance'] =
       (forecasted['fact'] - forecasted['yhat_upper'])/forecast['fact']
   forecasted.loc[forecasted['anomaly'] ==-1, 'importance'] =
       (forecasted['yhat_lower'] - forecasted['fact'])/forecast['fact']

   return forecasted
pred = detect_anomalies(pred)
```


Anomaly Detection
Sales
36,000
34,000
32,000
30,000
28,000
26,000
24,000
22,000
20,000
18,000
16,000
14,000
12,000
10,000
8,000
6,000
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
date